

Smart Energy Grid Monitoring System

Prepared by:

MUSABYIMANA Diane 101218

UGIZWENAYO Divine 101191

NDAHIMANA Raphael 101228

1. Infrastructure Overview

1.1 System Architecture

The Smart Energy Grid Monitoring System is designed to collect, store, and analyze energy consumption data from 1,000 simulated smart meters. The system leverages modern time-series database technology to demonstrate performance advantages of TimescaleDB's time-partitioning features.

The system consists of the following components:

- EMQX MQTT Broker: receives real-time meter readings via MQTT protocol on port 1883
- TimescaleDB: PostgreSQL-based time-series database storing 8+ million readings on port 5433
- Python Subscriber: listens to MQTT topics and persists data to TimescaleDB
- Python Generator: simulates 1,000 smart meters publishing data every 5 minutes
- Grafana Dashboard: visualizes energy data and performance metrics on port 3000
- Docker: containerizes all services for consistent deployment

1.2 Technology Stack

Component	Technology
MQTT Broker	EMQX
Time-series DB	TimescaleDB
Database Engine	PostgreSQL
Dashboard	Grafana
Containerization	Docker Desktop
Data Pipeline	Python + paho-mqtt

1.3 Data Schema

The `energy_readings` table stores all meter data with the following structure:

Column	Type	Description
<code>meter_id</code>	TEXT	10-digit unique meter identifier
<code>timestamp</code>	TIMESTAMPTZ	Time of reading (partition key)
<code>power</code>	DOUBLE PRECISION	Power consumption in watts
<code>voltage</code>	DOUBLE PRECISION	Voltage in volts
<code>current</code>	DOUBLE PRECISION	Current in amperes
<code>frequency</code>	DOUBLE PRECISION	Frequency in Hz
<code>energy</code>	DOUBLE PRECISION	Energy consumed in kWh

2. Implementation Details

2.1 Data Generation

We wrote a Python program that simulates 1,000 smart meters, each reporting energy readings every 5 minutes. To make the data realistic, power consumption varies by time of day:

- Morning (6:00–9:00 AM): 3.0–5.0W
- Evening peak (5:00–9:00 PM): 4.0–6.0W
- Night (10:00 PM–5:00 AM): 0.3–1.5W
- Daytime (9:00 AM–5:00 PM): 1.5–3.0W

For loading historical data, we quickly realized that running the real-time simulator for 4 actual weeks was not practical. Instead, we built a `fast_loader.py` script that generates past timestamps and inserts them as fast as possible.

Total data loaded: 8,093,000 rows covering 4 weeks of historical data (April 19 - May 17, 2026).

2.2 TimescaleDB Hypertable Configuration

The `energy_readings` table was converted to a TimescaleDB hypertable partitioned by timestamp with a 1-day chunk interval. Three configurations were tested:

- `energy_readings`: 1-day chunk interval (29 chunks for 28 days)
- `energy_readings_3h`: 3-hour chunk interval (225 chunks for 28 days)
- `energy_readings_week`: 1-week chunk interval (5 chunks for 28 days)

2.3 Continuous Aggregation Views

Three materialized views were created for pre-computed aggregations:

- energy_readings_15min: 15-minute buckets with AVG, MAX power, and SUM energy
- energy_readings_1h: 1-hour buckets with AVG, MAX power, and SUM energy
- energy_readings_1day: Daily buckets with AVG, MAX power, and SUM energy

3. Performance Analysis

3.1 Baseline Query Performance (1-Day Chunks)

All queries were executed with cold cache conditions (PostgreSQL restarted before each test):

Query	Description	Execution Time
Query 1	Average power per hour today	107 ms
Query 2	Peak consumption in past 7 days	357.683 ms
Query 3	Monthly consumption per meter	4,086.324 ms
Query 4	Full dataset scan (COUNT, AVG, MAX, MIN)	530.506 ms

3.2 Chunk Interval Comparison

The same four queries were executed on all three chunk configurations under cold cache conditions:

Query	3h Chunks	1-Day Chunks	1-Week Chunks
Query 1	267.418 ms	107 ms	60.411 ms
Query 2	730.307 ms	357.683 ms	843.282 ms
Query 3	6,214.986 ms	4,086.324 ms	3,216.277 ms
Query 4	617.511 ms	530.506 ms	753.862 ms

Key observations:

- Query 1: 1-week chunks is the fastest due to fewer chunks to scan for today's data
- Query 2: 1-day chunks is optimal due to balanced chunk granularity for weekly queries
- Query 3: 1-week chunks is the fastest due to fewer chunk metadata lookups for full scans
- Query 4: 1-day chunks is optimal due to balanced performance across all query types

3.3 Compression Analysis

3.3.1 Storage Efficiency

Hypertable	Before Compression	After Compression	Space Saved	Ratio
energy_readings (1-day)	783 MB	397 MB	386 MB	1.97x
energy_readings_3h	794 MB	413 MB	381 MB	1.92x
energy_readings_week	782 MB	395 MB	387 MB	1.98x

All three hypertables achieved approximately 2x compression ratio, saving nearly 50% of disk space.

3.3.2 Query Performance After Compression

Query	Before Compression	After Compression	% Change
Query 2	2,000.290 ms	208.620 ms	90% faster
Query 3	11,370.505 ms	16,517.890 ms	45% slower

Analysis:

- Query 2 improved dramatically after compression (nearly 10x improvement). This is because TimescaleDB's columnar compression stores data column by column, so a query that only reads the timestamp and power columns can skip the voltage, current, frequency, and energy columns entirely.
- Query 3 became slower: this query aggregates all meters across all months, requiring decompression of the entire dataset. The decompression overhead outweighs the I/O savings for this type of full-table analytical query. In a production system, we would address this by using continuous aggregates for Query 3 rather than querying the raw compressed table.

3.4 Continuous Aggregation Performance

The performance improvement from continuous aggregations was the most striking result of our entire analysis

Approach	Query Type	Execution Time	Improvement
Raw data query	15-min aggregation, 2-day range	60.411 ms	Baseline
Continuous aggregate view	15-min aggregation, 2-day range	6.374 ms	Approximately 10x faster

The continuous aggregate view is almost 10 times faster than querying the raw table. This makes a huge practical difference for a dashboard that might run this query dozens of times per minute across many users. By pre-computing the aggregations and storing the results, TimescaleDB eliminates the need to re-scan millions of rows for every dashboard refresh.

4. Dashboard Description

The Grafana dashboard (accessible at <http://localhost:3000>) includes 7 panels:

Panel	Title	Description
1	Real-time Meter Readings	Average power consumption over recent period, 5-minute buckets
2	Daily Consumption Patterns	Hourly average power showing daily peaks and troughs
3	Weekly Trends	Daily min/avg/max power over 4-week period
4	Monthly Usage by Region	Energy consumption grouped by meter region (first digit of ID)
5	Query Performance: Raw vs Aggregated	Bar chart comparing raw query vs continuous aggregate view times
6	Storage Efficiency: Compression	Before/after compression sizes for all three hypertables
7	Chunk Strategy Comparison	Side-by-side query times for 3h, 1-day, and 1-week chunk configs

5. Recommendations

5.1 Optimal Chunk Interval

Based on our testing, we recommend 1-day chunks as the default for energy monitoring workloads. This configuration provided the best overall balance across all four query types, and it aligns naturally with how energy data is typically reported and analyzed (daily usage reports, daily averages, etc.).

For deployments that primarily run analytical reports over long time periods, 1-week chunks offer better performance on full-table scans.

We would avoid 3-hour chunks for large datasets because the overhead of managing hundreds of small chunks outweighs any potential benefit.

5.2 Compression Strategy

- Enable compression for all data older than 24 hours because it achieves ~2x storage savings
- Use continuous aggregates for dashboard queries because performance improves 10 times.
- Keep recent data (last 24 hours) uncompressed for fast real-time writes

- For full-table analytical queries, consider disabling compression or using columnar storage

5.3 Continuous Aggregation Strategy

Continuous aggregations should be used for all dashboard panels. Our testing showed almost 10 times performance improvement compared to querying the raw table, and this improvement only becomes more significant as the dataset grows. We recommend creating at least 15-minute, hourly, and daily materialized views, and building a cascade where hourly views are derived from 15-minute views for maximum efficiency.

6. Conclusion

This project successfully demonstrates the advantages of TimescaleDB for time-series data management. Key findings include:

- TimescaleDB hypertables provide significant query performance improvements over regular PostgreSQL tables through automatic time-based partitioning
- Chunk interval selection has a measurable impact on query performance, but the optimal chunk size depends on the query pattern
- Compression achieves approximately 2 times storage reduction with mixed query performance impact, beneficial for range queries, slightly detrimental for full scans, making it a win for most deployments.
- Continuous aggregations provide the biggest performance improvement (almost 10 times) for dashboard queries and should be used wherever possible.
- The combination of hypertables, compression, and continuous aggregates makes TimescaleDB a powerful solution for IoT and energy monitoring applications

We successfully built a complete end-to-end pipeline: from data simulation through MQTT, to storage in TimescaleDB, to visualization in Grafana. The system processed 8,093,000 rows of smart meter data and demonstrated measurable, quantified performance differences across multiple configurations.